VoteFlix

Movie Awards Voting System

By: Group 11

Project Overview

Our project is a **Movie Awards Voting System**. It is a relational database application that models and manages the full lifecycle of an awards program, from movies and cast/crew to nominations, voting, and results.

Core Entities

Movies, People, Users, Nominations, Award Categories, and Votes — all interconnected through well-defined relationships.

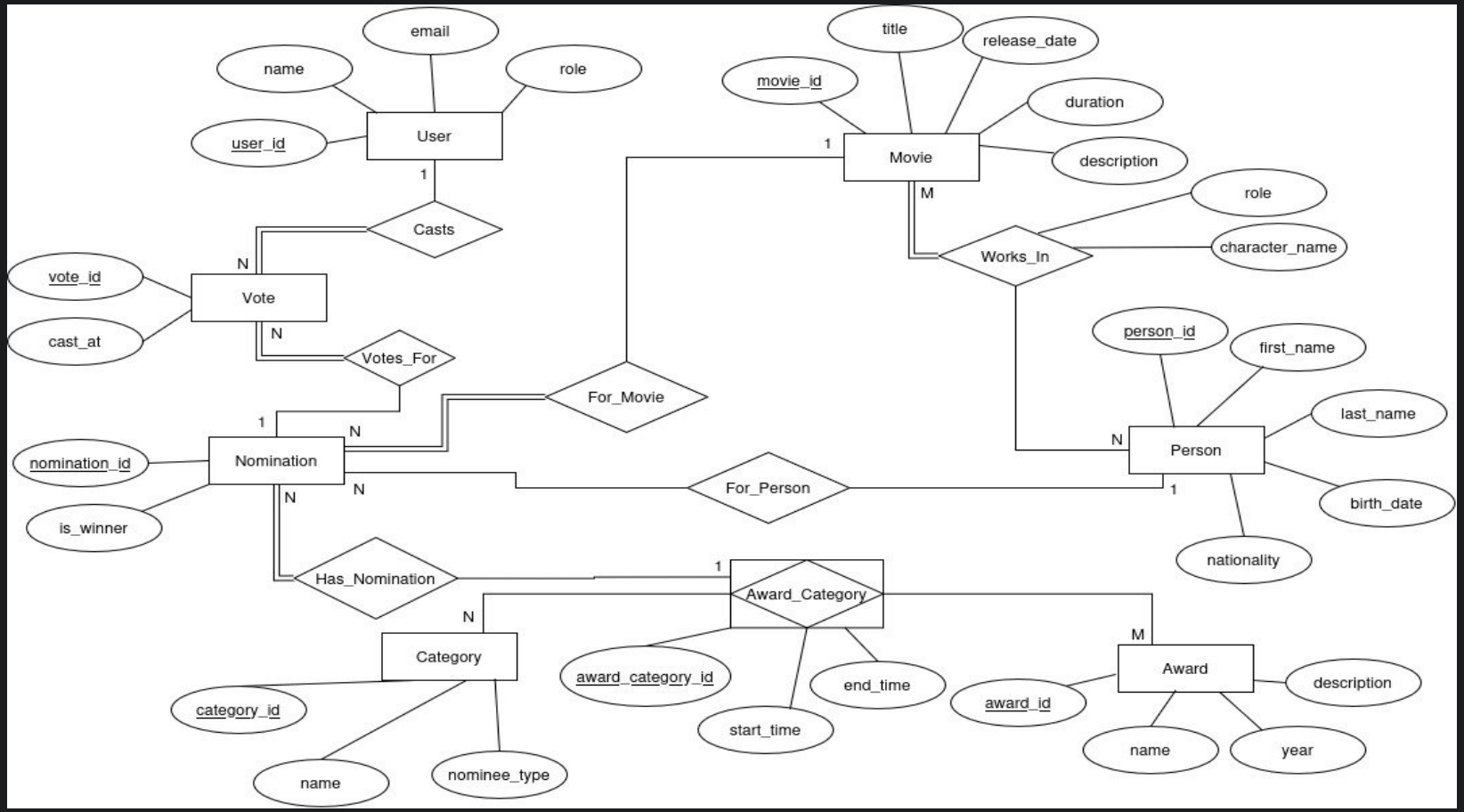
System Goals

Ensure data integrity, support efficient querying, and enforce fair voting logic through carefully structured schema design.

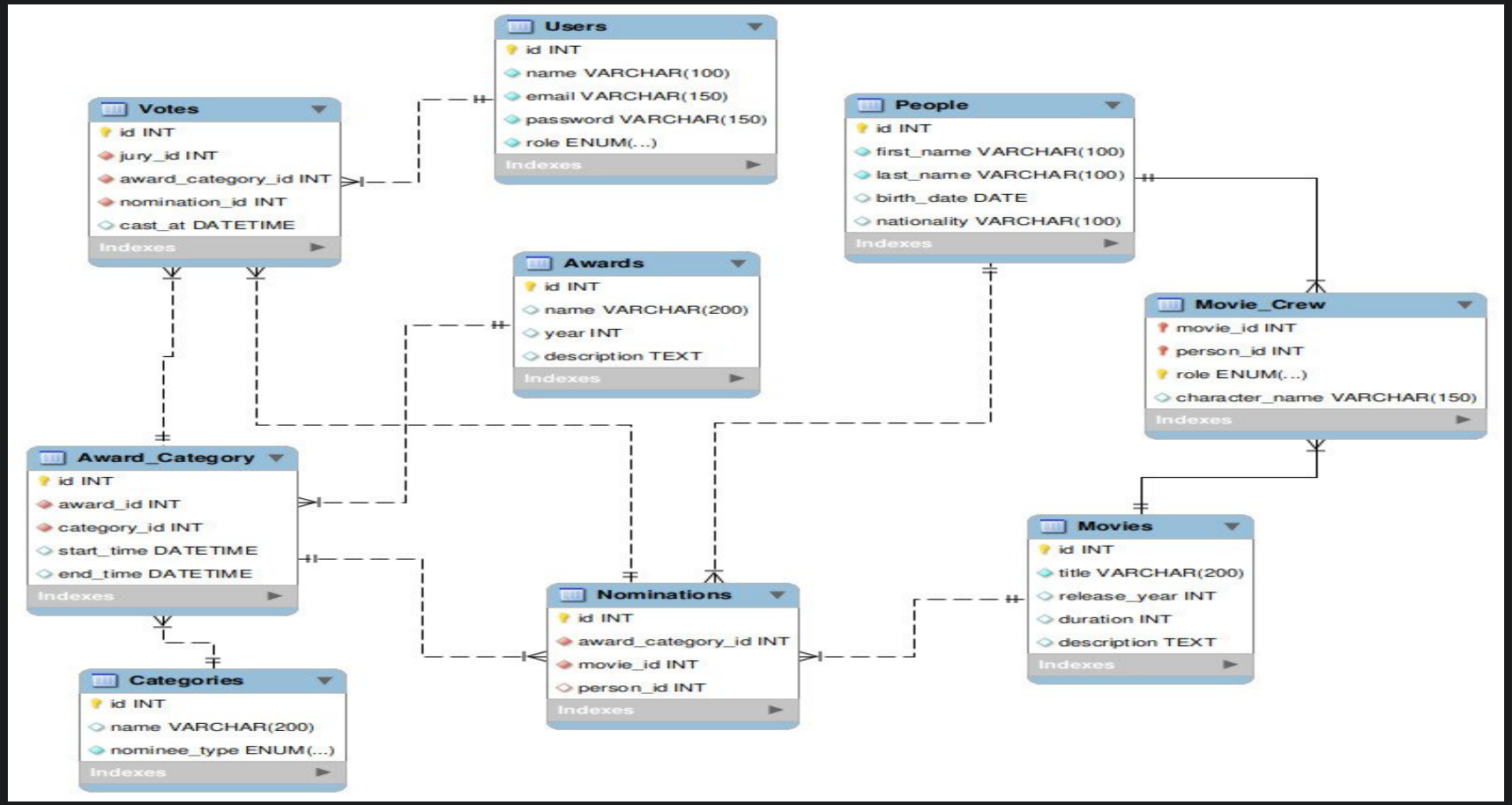
User Interaction

Users browse nominees and cast votes. The system aggregates results and calculates winners based on vote counts per nomination.

Entity-Relationship Diagram



Schema Design



Normalization

We applied all three standard normal forms to eliminate redundancy, prevent anomalies, and ensure data consistency throughout the schema.

1NF — Atomic Fields

Every column holds a single, indivisible value. No repeating groups or multi-valued attributes exist in any table.

2NF — No Partial Dependencies

All non-key attributes depend on the *entire* primary key. Composite keys in bridge tables like `Movie_Crew` were handled carefully.

3NF — No Transitive Dependencies

Movie data and Person data are stored in separate tables and linked via `Movie_Crew`, eliminating any transitive dependency between non-key columns.

SQL Joins

Movie Vote Aggregation

Aggregates votes across all nominations of a movie to determine its overall popularity and ranking in the system.

```sql
SELECT
  m.title,
  COUNT(v.id) AS vote_count
FROM Votes v
JOIN Nominations n ON v.nomination_id = n.id
JOIN Movies m ON n.movie_id = m.id
GROUP BY m.title
HAVING vote_count > 0;
```

Vote Summary

Used to display complete nomination info (movie + person + category + award)

```sql
SELECT
    n.id AS nomination_id,
    m.title AS movie,
    COALESCE(CONCAT(p.first_name, ' ', p.last_name), 'N/A') AS person,
    c.name AS category,
    a.name AS award,
    a.year
FROM Nominations n
JOIN Movies m ON n.movie_id = m.id
LEFT JOIN People p ON n.person_id = p.id
JOIN Award_Category ac ON n.award_category_id = ac.id
JOIN Categories c ON ac.category_id = c.id
JOIN Awards a ON ac.award_id = a.id;
```

Group By and Having

Category Statistics

Aggregates total nominations and votes for each category.

```sql
SELECT
    c.name,
    COUNT(DISTINCT n.id) AS total_nominations,
    COUNT(v.id) AS total_votes
FROM Categories c
LEFT JOIN Award_Category ac ON c.id = ac.category_id
LEFT JOIN Nominations n ON n.award_category_id = ac.id
LEFT JOIN Votes v ON v.nomination_id = n.id
GROUP BY c.id, c.name;
```

Winners View

Uses GROUP BY to find the highest vote count per category and select winners.

```sql
CREATE VIEW Winners AS
SELECT vs.*
FROM Vote_Summary vs
JOIN (
    SELECT category, MAX(total_votes) AS max_votes
    FROM Vote_Summary
    GROUP BY category
) t
ON vs.category = t.category
AND vs.total_votes = t.max_votes;
```

Sub-queries

Winners View

Subquery for Max Votes per Category, finds winner per category

```
CREATE VIEW Winners AS
SELECT vs.*
FROM Vote_Summary vs
JOIN (
    SELECT category, MAX(total_votes) AS max_votes
    FROM Vote_Summary
    GROUP BY category
) t
ON vs.category = t.category
AND vs.total_votes = t.max_votes;
```

Top Nominations

Finds nominations with maximum votes

```
SELECT nomination_id
FROM Votes
GROUP BY nomination_id
HAVING COUNT(*) = (
  SELECT MAX(cnt)
  FROM (
    SELECT COUNT(*) AS cnt
    FROM Votes
    GROUP BY nomination_id
  ) t
);
```

Views

Nomination_Details

Provides complete nomination details by combining movies, people, categories, and awards.

```sql
CREATE VIEW Nomination_Details AS
SELECT
    n.id AS nomination_id,
    m.title AS movie,
    COALESCE(CONCAT(p.first_name, ' ', p.last_name), 'N/A') AS person,
    c.name AS category,
    a.name AS award,
    a.year
FROM Nominations n
JOIN Movies m ON n.movie_id = m.id
LEFT JOIN People p ON n.person_id = p.id
JOIN Award_Category ac ON n.award_category_id = ac.id
JOIN Categories c ON ac.category_id = c.id
JOIN Awards a ON ac.award_id = a.id;
```

Vote_Summary

Calculates total votes for each nomination.

```sql
CREATE VIEW Vote_Summary AS
SELECT
    n.id AS nomination_id,
    m.title,
    c.name AS category,
    COUNT(v.id) AS total_votes
FROM Nominations n
JOIN Movies m ON n.movie_id = m.id
JOIN Award_Category ac ON n.award_category_id = ac.id
JOIN Categories c ON ac.category_id = c.id
LEFT JOIN Votes v ON v.nomination_id = n.id
GROUP BY n.id, m.title, c.name;
```

Stored Procedures & Transactions

CastVote

Provides complete nomination details by combining movies, people, categories, and awards.

```
CREATE PROCEDURE CastVote(
    IN p_jury INT,
    IN p_nomination_id INT,
    IN p_category_id INT
)
BEGIN
    DECLARE existing_vote INT;
    DECLARE v_category_id INT;
    START TRANSACTION;

    -- Lock nomination row
    SELECT award_category_id INTO v_category_id
    FROM Nominations
    WHERE id = p_nomination_id
    FOR UPDATE;

    -- Lock votes row
    SELECT id INTO existing_vote
    FROM Votes
    WHERE jury_id = p_jury
      AND award_category_id = p_category_id
    FOR UPDATE;

    -- If vote already exists, rollback; otherwise, insert vote
    IF existing_vote IS NOT NULL THEN
        ROLLBACK;
    ELSE
        INSERT INTO Votes (jury_id, nomination_id, award_category_id)
        VALUES (p_jury, p_nomination_id, p_category_id);

        COMMIT;
    END IF;
END//
```

Stored Procedures & Transactions

Create Nomination

Creates a nomination.

```
CREATE PROCEDURE CreateNomination(
    IN p_award_category_id INT,
    IN p_movie_id INT,
    IN p_person_id INT
)
BEGIN
    DECLARE cat_type VARCHAR(50);
    START TRANSACTION;
    -- Get category type
    SELECT c.nominee_type INTO cat_type
    FROM Award_Category ac
    JOIN Categories c ON ac.category_id = c.id
    WHERE ac.id = p_award_category_id
    FOR UPDATE;

    -- If category type is movie then person is null
    IF cat_type = 'MOVIE' THEN
        INSERT INTO Nominations (award_category_id, movie_id, person_id)
        VALUES (p_award_category_id, p_movie_id, NULL);

    ELSE
        -- Check if person role is valid for the category type
        IF EXISTS (
            SELECT 1
            FROM Movie_Crew mc
            WHERE mc.movie_id = p_movie_id
              AND mc.person_id = p_person_id
              AND (cat_type = 'ANY_CREW' OR mc.role = cat_type)
            FOR UPDATE
        ) THEN
            INSERT INTO Nominations (award_category_id, movie_id, person_id)
            VALUES (p_award_category_id, p_movie_id, p_person_id);

            COMMIT;
        END IF;
    END IF;
END //
```

Stored Procedures & Transactions

Update Nomination Transaction

Updates nomination details and synchronizes related votes atomically.

```
CREATE PROCEDURE UpdateNomination(
    IN p_nomination_id INT,
    IN p_award_category_id INT,
    IN p_movie_id INT,
    IN p_person_id INT
)
BEGIN
    DECLARE prev_category INT;
    START TRANSACTION;

    -- Lock nomination row
    SELECT id
    FROM Nominations
    WHERE id = p_nomination_id
    FOR UPDATE;

    SELECT award_category_id INTO prev_category
    FROM Nominations
    WHERE id = p_nomination_id
    FOR UPDATE;

    -- Update nomination with the data
    UPDATE Nominations
    SET award_category_id = p_award_category_id,
        movie_id = p_movie_id,
        person_id = p_person_id
    WHERE id = p_nomination_id;

    -- Select and update votes for the nomination accordingly
    SELECT id FROM Votes
    WHERE nomination_id = p_nomination_id
    FOR UPDATE;

    UPDATE Votes
    SET award_category_id = p_award_category_id
    WHERE nomination_id = p_nomination_id;

    COMMIT;
END //
```

Closing Thoughts

Relational databases power real-world voting.

This project demonstrates how a well-structured relational database can efficiently manage a real-world voting system: balancing accuracy, performance, and data integrity at every layer of the design.

Accuracy

Constraints, foreign keys, and validated schemas ensure every vote is counted exactly once and linked to a valid nomination.

Performance

Strategic indexing on high-traffic columns keeps query response times fast, even as the votes table grows under real election conditions.

Data Integrity

Relational normalization and referential constraints prevent orphaned records and guarantee consistent, trustworthy results across all reporting queries.

Thank You!